

PROGRAMACIÓN I · SEMANA 4

Iteración indefinida

Ciclo while · banderas · 5 ejercicios

Keneth Ruiz · kgruiz@ufm.edu

Programación I

¿Qué vemos hoy?

01

Repaso: for y range

Lo que ya sabemos

02

¿Definida o indefinida?

Dos formas de repetir

03

El ciclo while

Repetir según una condición

04

Las 3 reglas de oro

Para que el while funcione

05

El bucle infinito

El error clásico

06

Flags (banderas)

Controlar con un booleano

07

Errores comunes

Cómo evitarlos

08

Ejercicios

5 retos con solución

Repaso — for y range

Ya saben repetir con **for**: sirve cuando saben **cuántas veces** se repite algo.

```
for i in range(1, 6):  
    print(i)  
  
# imprime: 1 2 3 4 5
```

El for ya sabe el número de vueltas

El range(1, 6) le dice exactamente: «hacé 5 vueltas». No tiene que pensarlo: ya conoce el final .

¿Y si NO sabemos cuántas veces? Por ejemplo: seguir preguntándole algo al usuario hasta que diga no, o pedir una contraseña hasta que sea correcta. Para eso existe el **while**.

CONTROL DE FLUJO · CICLOS

¿Definida o indefinida?

¿Qué es iterar?

Iterar = repetir un proceso varias veces. La pregunta clave es: **¿sabemos cuántas veces?**

Sí sabemos → for

Iteración definida

- Hacer 10 lagartijas
- Leer las 12 páginas del capítulo
- Contar del 1 al 100

NO sabemos → while

Iteración indefinida

- Comer hasta llenarte
- Preguntar hasta que diga no
- Intentar la clave hasta atinarle

for vs while — lado a lado

for — sé cuántas vueltas

```
# Sé que son 5 vueltas
for i in range(5):
    print('Hola')
```

while — no sé cuántas vueltas

```
# Repito MIENTRAS se cumpla algo
respuesta = 's'
while respuesta == 's':
    respuesta = input('¿Otra? ')
```

En una frase: el **for cuenta** las vueltas; el **while vigila una condición** y se repite mientras esa condición sea verdadera.

ITERACIÓN INDEFINIDA

El ciclo while

El while — estructura

```
while condición:  
    # código que se repite  
    # mientras la condición sea True
```

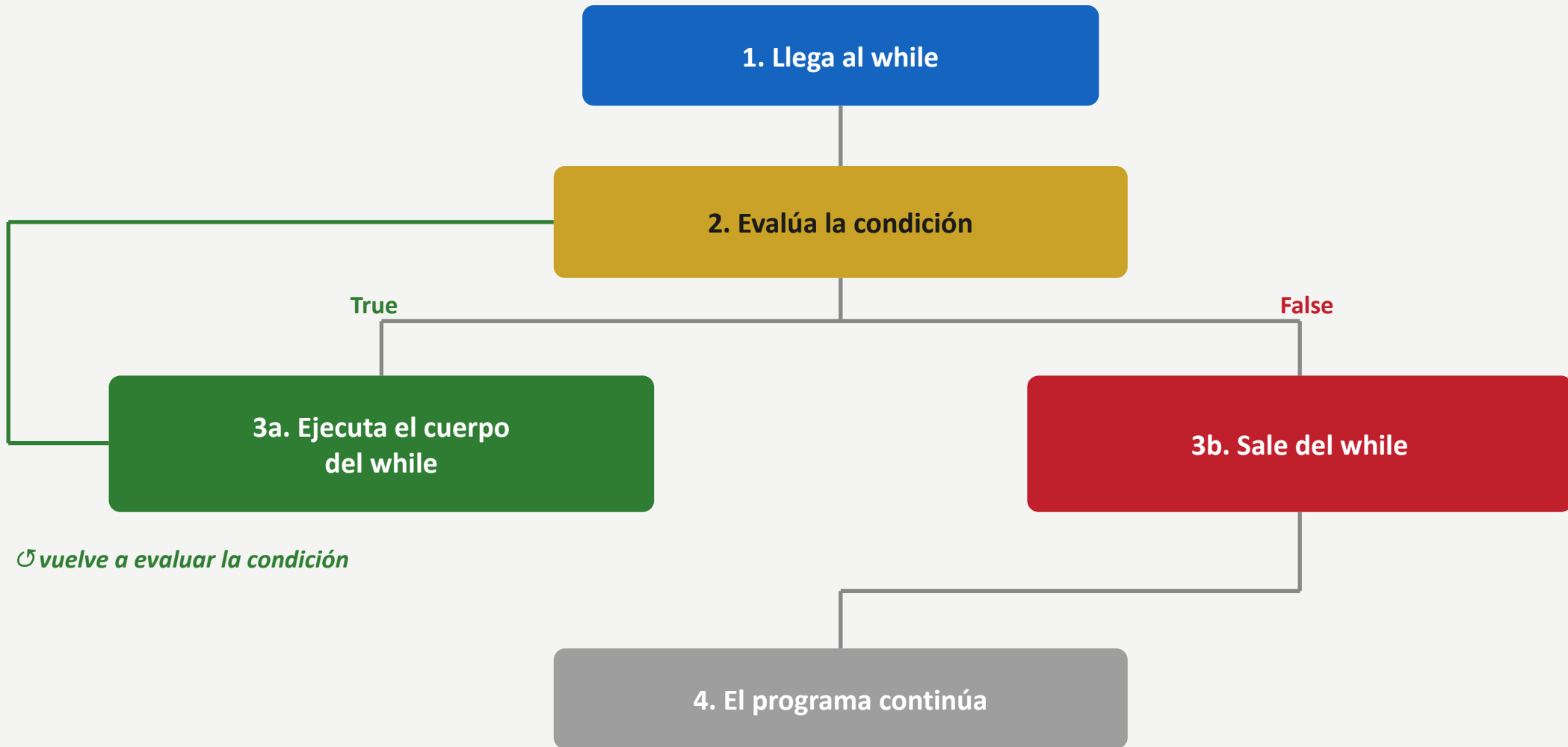
- 1 La palabra while — siempre en minúscula
- 2 La condición — algo que da True o False
- 3 Los dos puntos : — OBLIGATORIOS (igual que el if)
- 4 El cuerpo indentado con 4 espacios (igual que el if)

¿Te suena familiar?

Los dos puntos (:) y la indentación de 4 espacios son las MISMAS reglas que aprendiste con el if la semana antepasada.

Lo único nuevo es la palabra while y que el bloque se va a repetir, no a ejecutar una sola vez.

¿Qué hace Python con un while?



if vs while — la diferencia clave

if — decide UNA vez

```
edad = 16
if edad < 18:
    print('Menor')
print('Fin')
```

Pregunta una sola vez y sigue.

while — repite MUCHAS veces

```
n = 3
while n > 0:
    print(n)
    n = n - 1
print('Fin')
```

Pregunta, ejecuta y VUELVE a preguntar.

Mismo : y misma indentación. La única diferencia: el **while** **regresa** a revisar la condición; el if no.

Las 3 reglas CLAVE del while

1. INICIALIZAR

Creá la variable ANTES del ciclo.

```
contador = 5
```

2. CONDICIÓN

Una pregunta True/False que controla cuándo parar.

```
contador > 0
```

3. CAMBIAR

Modificá la variable DENTRO del ciclo para que algún día sea False.

```
contador = contador - 1
```



Si falta la regla #3 (cambiar la variable), el ciclo nunca termina → bucle infinito.

Primer ejemplo — cuenta regresiva

```
contador = 5
while contador > 0:
    print(contador)
    contador = contador - 1
print('¡Despegue!')
```

① inicializar

② condición

③ cambio de estado

Hacé la tabla a mano:

vuelta	contador	imprime
1	5	5
2	4	4
3	3	3
4	2	2
5	1	1
—	0	(sale)

Salida: 5 4 3 2 1 ¡Despegue!

El bucle infinito — el error clásico

❌ El error

```
contador = 5
while contador > 0:
    print(contador)
    # ¡falta cambiar contador!
```

✅ La solución

```
contador = 5
while contador > 0:
    print(contador)
    contador = contador - 1
```

contador SIEMPRE vale 5 → la condición SIEMPRE es True → el ciclo no para nunca.

💡 Si tu programa se cuelga, casi siempre es un bucle infinito. Para la ejecución con **Ctrl + C** en la terminal.

while con input — pedir hasta que sea válido

Un uso clásico del while: pedir datos una y otra vez hasta que sean correctos.

```
edad = int(input('Tu edad: '))
while edad < 0:
    print('La edad no puede ser negativa.')
    edad = int(input('Tu edad: '))
print('Tu edad es:', edad)
```

¡El while puede correr 0 veces!

Si la PRIMERA respuesta ya es válida (ej. 25), la condición es False de entrada y el cuerpo no se ejecuta ni una vez.

Patrón a recordar: pedís el dato ANTES del while, y lo volvéis a pedir DENTRO del while. Así, si está mal, vuelve a entrar; si está bien, sale.

ITERACIÓN INDEFINIDA

Flags (banderas)

¿Qué es un flag (bandera)?

Un flag es una variable booleana (True / False) que decide si el ciclo sigue o se detiene.



Como un semáforo: en verde (True) seguís; en rojo (False) parás.

```
seguir = True          # bandera en verde
while seguir:
    # hacer algo...
    # cuando quieras parar:
    seguir = False      # bandera en rojo
```

¿Cuándo conviene?

Cuando no querés contar vueltas, sino repetir «hasta que el usuario quiera parar»: menús, juegos, lanzar una moneda otra vez, etc.

ITERACIÓN INDEFINIDA

LOOP INFINITO

Loop Infinito Flag

```
while True:
    print('1')
```

Como la condición es siempre True, nunca se vuelve falsa → nunca se detiene → imprime 1 infinitamente. Por eso se llama "loop infinito".

1

Flag — ejemplo: lanzar una moneda

```
import random

seguir = True
while seguir:
    moneda = random.choice(['Cara', 'Cruz'])
    print('Salió:', moneda)
    r = input('¿Lanzar otra vez? (s/n): ')
    if r == 'n':
        seguir = False
print('¡Gracias por jugar!')
```

Paso a paso:

seguir = True

arranca activo (en verde)

while seguir:

repite mientras sea True

random.choice(...)

elige Cara o Cruz al azar

input(...)

el usuario decide si sigue

r == 'n' → False

la bandera pasa a rojo y termina

Herramientas extra — break y continue

break — salir YA del ciclo

```
n = 0
while True:
    n = n + 1
    if n == 5:
        break
    print(n)
# imprime: 1 2 3 4
```

continue — saltar a la siguiente vuelta

```
n = 0
while n < 5:
    n = n + 1
    if n == 3:
        continue
    print(n)
# imprime: 1 2 4 5
```

break corta el ciclo por completo; **continue** salta el resto de esa vuelta y vuelve a la condición. Útiles, pero usalos con cuidado para no perder de vista el flujo.

¿Cuándo uso for y cuándo while?

Usá for cuando...

- sabés cuántas veces repetir
- recorrés un rango fijo (0 a 100)
- contás algo de tamaño conocido

Usá while cuando...

- NO sabés cuántas veces
- dependés de una respuesta del usuario
- repetís «hasta que» pase algo

Regla simple: ¿puedo contar las vueltas de antemano? Sí → for. No → while.

PRACTIQUEMOS · INTENTALO PRIMERO

Ejercicios

Ejercicio 1 — Cuenta regresiva

 15 min

Pedí un número n al usuario y mostrá una cuenta regresiva desde n hasta 0.

- 1 Pedí n con `int(input(...))`.
- 2 Creá un contador = n antes del ciclo.
- 3 while contador ≥ 0 : mostralo.
- 4 Restale 1 en cada vuelta.
- 5 Al final imprimí ¡Listo!

Caso de prueba

```
Ingresa un número: 3
3
2
1
0
¡Listo!
```

 **Pista:** ojo con ≥ 0 para que el 0 también salga.

 **Bonus:** mostrá solo los números pares (usá %).

Solución — Ejercicio 1

Intentalo primero. Esta slide es para revisar después.

```
n = int(input('Ingresá un número: '))
contador = n
while contador >= 0:
    print(contador)
    contador = contador - 1
print('¡Listo!')
```

Nota: contador empieza en n y baja de 1 en 1 hasta que la condición `contador >= 0` se vuelve False.

Casos de prueba

```
n = 3
3 2 1 0
¡Listo!
```

```
n = 0
0
¡Listo!
```

Ejercicio 2 — Saltos de tres

 15 min

Imprimí todos los múltiplos de 3 que hay entre 0 y 100.

- 1 Creá numero = 0 antes del ciclo.
- 2 while numero <= 100: mostrá numero.
- 3 Sumá 3 en cada vuelta.
- 4 (numero = numero + 3)

Caso de prueba

```
0
3
6
9
...
96
99
```



Pista: arrancá en 0 y sumá de 3 en 3; el último que cabe es 99.



Bonus: contá cuántos múltiplos hay y mostralo al final.

Solución — Ejercicio 2

Intentalo primero. Esta slide es para revisar después.

```
numero = 0
while numero <= 100:
    print(numero)
    numero = numero + 3
```

Nota: no usamos un if con %: al sumar de 3 en 3 ya solo tocamos múltiplos de 3. 99 es el último ≤ 100 .

Casos de prueba

Salida

```
0 3 6 9 12 ...
... 93 96 99
```

Total

son 34 números

Ejercicio 3 — Suma con centinela

 20 min

Pedí números y sumalos. Cuando el usuario ingrese 0, mostrará el total y terminará.

- 1 Creá total = 0.
- 2 Pedí el primer numero (int(input...)).
- 3 while numero != 0: sumalo al total.
- 4 Volvé a pedir otro numero dentro.
- 5 Al salir, imprimí el total.

Caso de prueba

```
Número: 5  
Número: 3  
Número: 10  
Número: 0  
La suma total es: 18
```



Pista: el 0 es la «señal de parar» (centinela). No lo sumes.



Bonus: mostrará también cuántos números se ingresaron.

Solución — Ejercicio 3

Intentalo primero. Esta slide es para revisar después.

```
total = 0
numero = int(input('Número (0=fin): '))
while numero != 0:
    total = total + numero
    numero = int(input('Número (0=fin): '))
print('La suma total es:', total)
```

Casos de prueba

Entradas: 5, 3, 10, 0

La suma total es: 18

Nota: pedimos el número en DOS lugares (antes y dentro del while). Es el patrón típico del centinela.

Ejercicio 4 — Coin flip con flag

 20 min

Lanzá una moneda. Mostrá el resultado y preguntá si quiere lanzar de nuevo.

- 1 import random al inicio.
- 2 Creá el flag seguir = True.
- 3 while seguir: elegí Cara/Cruz y mostralo.
- 4 Preguntá «¿otra? (s/n)».
- 5 Si responde 'n', poné seguir = False.

 **Pista:** el flag empieza en True y solo cambia a False cuando el usuario quiere parar.

Caso de prueba

```
Salió: Cara
¿otra? (s/n): s
Salió: Cruz
¿otra? (s/n): n
¡Gracias por jugar!
```

 **Bonus:** validá que la respuesta sea 's' o 'n' (coin flip v2).

Solución — Ejercicio 4

Intentalo primero. Esta slide es para revisar después.

```
import random

seguir = True
while seguir:
    moneda = random.choice(['Cara', 'Cruz'])
    print('Salió:', moneda)
    r = input('¿Lanzar de nuevo? (s/n): ')
    if r == 'n':
        seguir = False
print('¡Gracias por jugar!')
```

Casos de prueba

Una corrida posible

```
Salió: Cara
Salió: Cruz
¡Gracias por jugar!
```

Nota: para el bonus (v2), envolví el input en otro while que repita mientras r no sea 's' ni 'n'.

Ejercicio 5 — Adivina el número

 25 min

La compu elige un número del 1 al 10. El usuario adivina hasta acertar.

1 `import random; secreto = random.randint(1, 10).`

2 Pedí el primer intento.

3 `while intento != secreto:`

4 decí si es muy alto o muy bajo.

5 Pedí otro intento; al acertar, felicitá.

Caso de prueba

```
secreto = 7
Adiviná: 5 → Muy bajo
Adiviná: 9 → Muy alto
Adiviná: 7 → ¡Correcto!
```

 **Pista:** combina `while` + `if/else` + `random`: ¡todo lo que ya saben!

 **Bonus:** contá los intentos y mostralos al final.

Solución — Ejercicio 5

Intentalo primero. Esta slide es para revisar después.

```
import random

secreto = random.randint(1, 10)
intento = int(input('Adiviná (1-10): '))
while intento != secreto:
    if intento < secreto:
        print('Muy bajo. Probá de nuevo.')
    else:
        print('Muy alto. Probá de nuevo.')
    intento = int(input('Adiviná (1-10): '))
print('¡Correcto! Era', secreto)
```

Casos de prueba

secreto = 7

5 → Muy bajo

9 → Muy alto

7 → ¡Correcto! Era 7

Nota: el while compara con != secreto; adentro un if/else dice la pista. Junta los 3 temas del curso.

ANTES DE CERRAR

¿Qué aprendieron hoy?

Lo que aprendieron hoy

- La iteración indefinida repite SIN saber cuántas veces de antemano.
- while repite MIENTRAS la condición sea True.
- Mismos dos puntos (:) e indentación que el if.
- Todo while necesita: inicializar, condición y cambio de estado.
- Sin cambio de estado → bucle infinito (pará con Ctrl + C).
- Un flag (booleano) controla cuándo seguir o parar.
- break sale del ciclo; continue salta a la siguiente vuelta.
- ¿Sé cuántas vueltas? for. ¿No sé? while.

¿Dudas?

Próxima clase: listas y cómo recorrerlas con for

Gracias :)